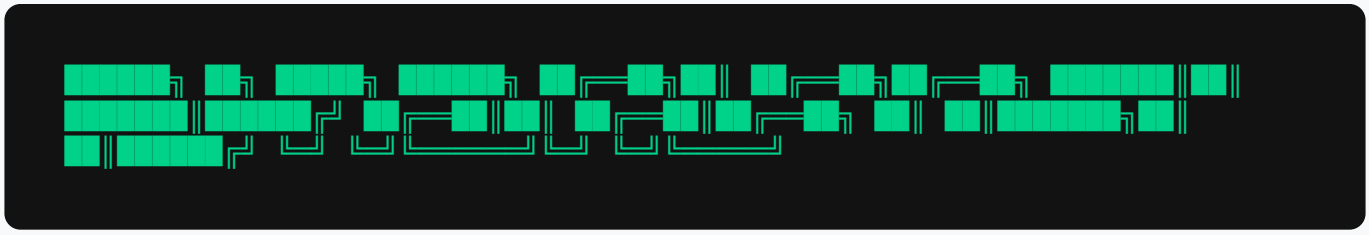
alab

Android Pentest Framework



v2.0 · Pixel 6 · API 33 · Magisk 25.2 + Zygisk · Frida 17.2.14 · KVM-accelerated

A no-bullshit Android pentesting framework. One command boots a rooted Pixel 6 emulator with Magisk + Zygisk + DenyList, pushes Frida, installs the Burp CA as a system cert, routes traffic through Burp, and exposes a clean CLI for unpinning, decompiling, and intercepting any APK.

Developed by hits

Built by human + AI (Opus 4.6)

github.com/hits313/alab

1 · Requirements

Component	Minimum	Notes
OS	Linux / macOS / Windows	All three covered below (WSL2 or native for Win)
CPU	x86_64 with VT-x/AMD-V (or Apple Silicon)	Hardware virtualization mandatory
RAM	8 GB system	AVD uses 3 GB, Burp + Frida add ~1 GB
Disk	25 GB free	SDK ~6 GB · system image ~5 GB · AVD ~10 GB
Python	3.10+	For Frida tools, objection, apkleaks
Java	OpenJDK 17	For jadx, apktool
Burp Suite	Community or Pro	Needed for proxy + cert export
Node.js	20+ (Optional)	For CLI agents (Claude Code / Gemini CLI)

2 · Install — Linux

Tested on Ubuntu 22.04 / Pop!_OS / Zorin OS / Debian 12. Other distros adjust `apt` → `dnf` / `pacman`.

2.1 · Pre-flight

```
ls /dev/kvm                           # must exist
egrep -c '(vmx|svm)' /proc/cpuinfo    # > 0
free -g                               # ≥ 8
df -h ~                               # ≥ 25 G
```

> **Missing KVM?**
>
> If `/dev/kvm` is missing, run:
> `sudo apt install qemu-kvm libvirt-daemon-system`
> `sudo usermod -aG kvm,libvirt $USER` (Log out and log back in).

2.2 · System packages

```
sudo apt update
sudo apt install -y   openjdk-17-jdk python3-pip python3-venv   qemu-kvm adb scrcpy
unzip wget curl git   apktool sqlite3 openssl
```

2.3 · Android SDK

We use command-line tools only — no Android Studio bloat.

```
mkdir -p ~/Android/Sdk/cmdline-tools && cd /tmp
wget https://dl.google.com/android/repository/commandlinetools-
linux-11076708_latest.zip
unzip commandlinetools-linux-*.zip
mv cmdline-tools ~/Android/Sdk/cmdline-tools/latest

export ANDROID_SDK_ROOT=$HOME/Android/Sdk
export PATH=$ANDROID_SDK_ROOT/cmdline-tools/latest/bin:$ANDROID_SDK_ROOT/platform-
tools:$ANDROID_SDK_ROOT/emulator:$PATH

yes | sdkmanager --licenses
sdkmanager "platform-tools" "emulator" "platforms;android-33"           "system-
images;android-33;google_apis;x86_64"
```

> **CRITICAL: Use google_apis**
>
> Use the `google_apis` image — NEVER `google_apis_playstore`. PlayStore images block `adb root`, breaking the entire toolchain. This is the #1 setup failure.

2.4 · Create AVD

```
echo "no" | avdmanager create avd   -n Pixel6_API33_root   -k "system-
images;android-33;google_apis;x86_64"   -d "pixel_6"

AVD=~/.android/avd/Pixel6_API33_root.avd/config.ini
sed -i 's/^hw.ramSize=.*/hw.ramSize=3072/' $AVD
sed -i 's/^hw.cpu.ncore=.*/hw.cpu.ncore=6/' $AVD
echo "disk.dataPartition.size=4096M" >> $AVD
```

2.5 · Frida + Tooling

```
pip install --user frida-tools==17.2.14 objection apkleaks androguard

mkdir -p ~/tools/android-lab/frida && cd ~/tools/android-lab/frida
wget https://github.com/frida/frida/releases/download/17.2.14/frida-server-17.2.14-
android-x86_64.xz
xz -d frida-server-*.xz && mv frida-server-* frida-server && chmod +x frida-server
```

2.6 · jadx + dex2jar + Magisk

```
cd ~/tools/android-lab
wget https://github.com/skylot/jadx/releases/download/v1.5.0/jadx-1.5.0.zip
unzip jadx-1.5.0.zip -d jadx

wget https://github.com/pxb1988/dex2jar/releases/download/v2.4/dex-tools-v2.4.zip
unzip dex-tools-v2.4.zip -d dex2jar

mkdir -p magisk && cd magisk
wget -O Magisk-v28.1.apk https://github.com/topjohnwu/Magisk/releases/download/v28.1/
Magisk-v28.1.apk
```

2.7 · Clone alab + Shell Aliases

```
git clone https://github.com/hits313/alab.git ~/tools/android-lab/alab
cp ~/tools/android-lab/alab/start-lab.sh ~/tools/android-lab/
chmod +x ~/tools/android-lab/start-lab.sh
```

Add the following to your `~/.zshrc` (or `~/.bashrc`):

```
export ANDROID_SDK_ROOT=$HOME/Android/Sdk
export PATH=$ANDROID_SDK_ROOT/platform-tools:$ANDROID_SDK_ROOT/emulator:$HOME/.local/
bin:$HOME/tools/android-lab/jadx/bin:$PATH

alias alab='bash ~/tools/android-lab/start-lab.sh'
```

3 · Install — macOS

Tested on macOS 14 Sonoma (Apple Silicon + Intel). HVF replaces KVM — works out of the box.

3.1 · Homebrew base

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/
install.sh)"

brew install openjdk@17 python@3.11 wget unzip git              android-platform-
tools scrcpy sqlite openssl@3
brew install --cask android-commandlinetools

sudo ln -sfn $(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk              /
Library/Java/JavaVirtualMachines/openjdk-17.jdk
```

3.2 · Android SDK

```
export ANDROID_SDK_ROOT=$HOME/Library/Android/sdk
mkdir -p $ANDROID_SDK_ROOT
export PATH=$(brew --prefix android-commandlinetools)/bin:$ANDROID_SDK_ROOT/platform-
tools:$ANDROID_SDK_ROOT/emulator:$PATH

yes | sdkmanager --licenses
# Apple Silicon → arm64-v8a  |  Intel → x86_64
ARCH=$(uname -m); [[ "$ARCH" == "arm64" ]] && IMG="arm64-v8a" || IMG="x86_64"
sdkmanager "platform-tools" "emulator" "platforms;android-33"            "system-
images;android-33;google_apis;$IMG"
```

> **Apple Silicon Pitfall**
>
> Always use the `arm64-v8a` system image on M-series Macs. Using `x86_64` under Rosetta translation will run at roughly 5% of native speed, rendering it unusable for dynamic analysis.

3.3 · Create AVD

```
echo "no" | avdmanager create avd   -n Pixel6_API33_root   -k "system-
images;android-33;google_apis;$IMG"   -d "pixel_6"
```

3.4 · Frida + Tooling

```
pip3 install --user frida-tools==17.2.14 objection apkleaks androguard

# frida-server — match device arch (emulator matches host arch)
mkdir -p ~/tools/android-lab/frida && cd ~/tools/android-lab/frida
ARCH=$(uname -m); [[ "$ARCH" == "arm64" ]] && FA="arm64" || FA="x86_64"
wget https://github.com/frida/frida/releases/download/17.2.14/frida-server-17.2.14-
android-$FA.xz
xz -d frida-server-*.xz && mv frida-server-* frida-server && chmod +x frida-server
```

3.5 · Clone alab + ZSH Aliases

```
mkdir -p ~/tools && git clone https://github.com/hits313/alab.git ~/tools/android-lab
chmod +x ~/tools/android-lab/start-lab.sh

cat >> ~/.zshrc <<'EOF'
export ANDROID_SDK_ROOT=$HOME/Library/Android/sdk
export PATH=$ANDROID_SDK_ROOT/platform-tools:$ANDROID_SDK_ROOT/emulator:$HOME/.local/
bin:$HOME/tools/android-lab/jadx/bin:$PATH
alias alab='bash ~/tools/android-lab/start-lab.sh'
EOF
source ~/.zshrc
```

4 · Install — Windows

Two paths are available. WSL2 is **strongly recommended** as it mirrors the Linux flow with minimal friction. Native Windows is supported but entails more environmental troubleshooting.

4.1 · Option A: WSL2 (Recommended)

```
# In elevated PowerShell
wsl --install -d Ubuntu-22.04
wsl --update
```

Inside WSL2, follow the standard Linux flow (§2). One extra detail requires attention:

```
# WSL2 needs explicit KVM passthrough for the emulator.
# First, enable Hyper-V on the Windows side, then in WSL2:
sudo apt install -y qemu-kvm
ls /dev/kvm   # should exist on WSL2 kernel 5.10+
```

> **WSL2 Networking**
>
> `adb` from WSL2 talks natively to the AVD running inside WSL2. To capture traffic using Burp on the Windows host:
> `ip route show | grep default | awk '{print $3}' # → 172.x.x.1 (Windows Host IP)`
> Point the AVD proxy at this IP, and set Burp to listen on `0.0.0.0:8080`.

4.2 · Option B: Native Windows

```
# Install prerequisites via winget
winget install -e --id EclipseAdoptium.Temurin.17.JDK
winget install -e --id Python.Python.3.11
winget install -e --id Git.Git
winget install -e --id 7zip.7zip
```

Download cmdline-tools and unzip to `C:\Android\Sdk\cmdline-tools\latest\`. Then set your environment variables:

```
ANDROID_SDK_ROOT  =  C:\Android\Sdk
PATH              += C:\Android\Sdk\cmdline-tools\latest in
PATH              += C:\Android\Sdk\platform-tools
PATH              += C:\Android\Sdk\emulator
```

In a **new** PowerShell window:

```
sdkmanager --licenses
sdkmanager "platform-tools" "emulator" "platforms;android-33" `
           "system-images;android-33;google_apis;x86_64"

echo no | avdmanager create avd -n Pixel6_API33_root `
  -k "system-images;android-33;google_apis;x86_64" -d "pixel_6"

py -m pip install --user frida-tools==17.2.14 objection apkleaks androguard
git clone https://github.com/hits313/alab.git C:ools ndroid-lab
```

Windows Gotchas

- **Virtualization:**

  Win 11 uses Hyper-V (built-in). Older Win 10 requires Intel HAXM via SDK manager.

- **Git Bash:**

  `start-lab.sh` is a Bash script. Run it via Git Bash, *not* cmd.exe or PowerShell.

- **Antivirus:**

  Defender will likely flag `frida-server.exe` and the Magisk APK. Allow-list the `C: ools ndroid-lab\` directory immediately.

5 · Quick Start

Four commands to achieve a fully wired lab environment.

```
$ alab start    # Boot AVD · auto-root · proxy · zygisk
$alab setup    # Full chain setup: root + frida + burp-cert + proxy$ alab status   #
Verifies device · root · magisk · frida · proxy status
$ alab stop     # Kill emulator instance
```

Burp Wiring (One-Time Setup)

1. In Burp, navigate to Proxy → Proxy Settings → Import/Export CA → **DER format**.
2. Save the exported file to `~/tools/android-lab/burp/cacert.der`.
3. Run `alab burp-cert` (This converts the cert, pushes it as a system-level CA, and reboots).
4. Run `alab proxy-on`.

Result: All HTTP/S traffic flows cleanly through Burp. No app-level proxy configuration required. No user-store certificate dance.

6 · Command Reference

6.1 · Boot & System

Command	Action

`alab start`	Boot AVD, auto root, enable proxy, Zygisk, and DenyList.
`alab stop`	Kill emulator process cleanly.
`alab screen`	Launch scrcpy mirror (1080p, screen-off on host).

6.2 · Root & Magisk

Command	Action
`alab root`	Invoke `adb root` and remount `/system` as writable.
`alab setup`	Execute full chain: root + frida + proxy + magisk.
`alab magisk`	Show Magisk daemon version and DenyList status.
`alab denylist`	Enable Zygisk + DenyList via internal sqlite3 DB mod.
`alab denylist-add com.x.y`	Add target package to Magisk DenyList (Root hiding).

6.3 · Intercept

Command	Action
`alab frida`	Push architecture-matched frida-server and start it.
`alab frida-stop`	Kill frida-server background process.
`alab burp-cert`	Install Burp CA as a system-trusted certificate + reboot.
`alab proxy-on`	Route all device HTTP/S traffic → `10.0.2.2:8080` (Burp).
`alab proxy-off`	Clear global proxy settings.

6.4 · SSL Unpinning

Command	Action
`alab unpin com.x.y`	Use `objection` to spawn app and run `android sslpinning disable`.
`alab unpin-frida com.x.y`	Universal Frida hook covering OkHttp3, TrustKit, TM, WebView, NSC.

6.5 · APK Analysis

Command	Action
`alab pull-apk com.x.y`	Extract the base APK for an installed package to the host.
`alab decompile app.apk`	Run `jadx` to decompile the APK into `/tmp/jadx-<name>/`.
`alab strings app.apk`	Run `apkleaks` to aggressively dump hardcoded secrets/endpoints.
`alab manifest app.apk`	Dump and decode `AndroidManifest.xml` using apktool.

7 · Attaching a CLI Agent

The lab is fully scriptable. Both Claude Code and the Gemini CLI can act as agentic pentesting drivers, utilizing `alab` verbs end-to-end for reconnaissance, unpinning, decompilation, and interception.

7.1 · Claude Code

```
npm i -g @anthropic-ai/claude-code
claude --version
cd ~/tools/android-lab && claude
```

Pre-grant `alab` execution permissions by writing this to `~/tools/android-lab/.claude/settings.json`:

```
{
  "permissions": {
    "allow": [
      "Bash(alab *)", "Bash(adb *)", "Bash(frida *)",
      "Bash(frida-ps *)", "Bash(objection *)", "Bash(jadx *)",
      "Bash(apktool *)", "Bash(apkleaks *)"
    ]
  }
}
```

7.2 · Gemini CLI

```
npm i -g @google/gemini-cli
gemini auth login
cd ~/tools/android-lab && gemini

> Use the alab framework at start-lab.sh. Understand the commands,
  then boot the emulator, install /tmp/target.apk, and decompile
  it with jadx.
```

8 · Troubleshooting

Issue	Fix
Emulator slow / black screen	Verify KVM/HVF virtualization. Ensure user is in `kvm` group (Linux). Fallback flag: `-gpu swangle`.
`adb root` **fails**	The system image must be exactly `google_apis`. Do NOT use `google_apis_playstore`.
Frida version mismatch	Ensure host `frida --version` identically matches the pushed `frida-server` binary version.
Burp cert not trusted	Re-run `alab burp-cert`. System cert installation strictly requires `-writable-system` mounting.
App detects root & exits	Run `alab denylist-add com.target.app`, then use `objection root-disable` hooks if necessary.

Issue	Fix
TLS still blocked / Handshake fails	Run `alab unpin-frida <pkg>` to apply the universal unpin script (handles OkHttp3, TrustKit, etc.).
WSL2 `/dev/kvm` missing	Enable Hyper-V in Windows Features, then `sudo apt install qemu-kvm` inside WSL2.

9 · Layout

Repo structure where all internal binaries and scripts reside:

```
~/tools/android-lab/
│
├── start-lab.sh                # alab dispatcher
├── unpin.sh                    # SSL unpin wrapper
├── magisk-root.sh              # Magisk install helper
├── frida/
│   └── frida-server            # arch-matched binary
├── frida-scripts/
│   └── universal-ssl-unpin.js  # OkHttp3 + TrustKit + TM + WebView
├── jadx/bin/jadx               # Java decompiler
├── dex2jar/dex2jar/d2j-dex2jar.sh
├── magisk/Magisk-v28.1.apk
├── burp/
│   ├── cacert.der              # Exported directly from Burp
│   └── cacert.pem              # Converted & auto-generated hash
├── apks/                       # Drop analysis targets here
└── docs/
    ├── ALAB-INSTALL.md         # Source markdown
    └── ALAB-INSTALL.pdf        # Generated document
```